

Comparing RNA models for something something

Bryan Richlinski *University of Kansas*

brichli@ku.edu Dominic Tassio *University of Kansas*

itsamemario@ku.edu Joseph Joestar *University of Kansas*

bobsyouruncle@ku.edu

Abstract

results indicate the entropy-ordered enumeration is a promising general-purpose technique to accelerate goal-directed top-down search for program synthesis.

I. INTRODUCTION

RNA molecules can be grouped into families based on shared characteristics such as structural patterns, sequence similarities, and functional roles. This allows researchers to study them in a more organized and meaningful way. To capture these similarities, a technique known as Multiple Read Alignment (MRA) is often employed, in which several RNA sequences from the same family are aligned together. From this alignment, a model can be constructed that encodes the common features of the family. This model can then be used as a reference to evaluate new RNA reads, providing a measure of how likely it is that a given sequence belongs to the same RNA family.

The chosen model for representing RNA families has often been a Stochastic Markov Model (SMM), which is capable of describing many RNA molecules in terms of both their raw symbolic strings and certain structural configurations that the molecules can take. This model has the same expressive power as a Stochastic Regular Grammar (SRG), meaning it can capture a wide range of sequence and structural patterns. However, it is limited in that it cannot represent more complex structures. These more intricate configurations are referred to as Secondary Structures (SS), and they require more powerful modeling techniques to be fully captured. To address this issue, a Covariance Model (CM) is often used to model these structures. The CM is a form of restricted Stochastic Context Free Grammar (SCFG) which captures a subset of the language of nested strings, the same set represented by Stochastic Visibly Pushdown Automaton (SVPA).

As we are primarily concerned with secondary structural properties, we choose to use the latter technique when comparing different RNA families. We hypothesize that RNA families themselves can be examined for similarity, by comparing the similarity of the models used to represent them. We propose a modified tree edit distance for examining the similarity of different Covariance Models which compares both structure and log-odd transitions.

We validate our algorithm as finding the smallest edit distance between two Covariance Models in Section II. To validate the efficacy of our approach we compare our results to the clan of RNA families. These clans encode how similar families are to each other based upon empirical evidence. By comparing the distance between models, to their respective clan we infer a cutoff and validate upon a hold out set. We evaluate this approach in Section V.

II. FORMALISM

A. Zhang-Shasha Algorithm

In our approach to the problem, we decided to use the Zhang-Shasha algorithm for tree edit distance. The Zhang-Shasha algorithm (published by Kaizhong Zhang and Dennis Shasha in 1989) is a dynamic programming algorithm that computes tree edit distance for ordered, labeled trees. These terms are straightforward: in labeled trees, nodes are assigned one of a selection of distinct types (labels), and exchanging two nodes may produce a different tree, if they are not of the same type. Ordered trees, as the name implies, rely on a fixed (ordered) interpretation of their children, usually left to right. Again, swapping a left child and a right child may produce a distinct tree, depending on their own labels and descendants. Fortunately, this fits our scenario well. The nodes in the trees we consider have distinct, meaningful contents, so labels are appropriate. Since the models are RNA-derived, and RNA is read left to right, the notion of order should be applicable as well. There are also practical considerations, as according to Akutsu et. al. (2009), in further work, Zhang and Shasha proved that the general case of tree edit distance for unordered trees is NP-hard.

As a dynamic programming algorithm, the Zhang-Shasha algorithm bears some similarity to other dynamic programming algorithms, such as string edit distance. To calculate tree edit distance, the minimal cost is calculated to transform one tree into another using three basic operations: adding, deleting, or re-labeling a node. However, as trees allow arbitrary connections and are more complicated than strings, additional methods are needed. Two phases are needed to calculate tree edit distance. A dynamic programming table is built up, calculating the cost of changing any sub-tree in the input to any sub-tree in the output, and the results combined for the total tree edit distance, and for each sub-tree, the authors generalize the algorithm to forest edit distance.

By numbering the nodes in each (sub)tree by performing a postorder traversal, children are given lower numbers than their parents, with the leftmost descendant of the root receiving the number 1 and the root being assigned some value n . By selecting

Algorithm 1 The Zhang-Shasha Algorithm

Require: Initial tree T_1 and target tree T_2

▷ Setup/Preprocessing

```

1:  $LRK_1 \leftarrow$  all keyroots of  $T_1$ 
2:  $LRK_2 \leftarrow$  all keyroots of  $T_2$ 
3: for each node  $i$  in  $T_1$  and  $T_2$  do
4:    $l[i] =$  leftmost descendant of  $i$ 
5: end for

```

▷ Main Algorithm

```

6: for  $i$  in  $LRK_1$  do
7:   for  $j$  in  $LRK_2$  do
8:      $treedist[i][j] = \text{find\_treedist}(i, j)$ 
9:   end for
10: end for
11: return  $treedist[i][j]$ 

```

a range $[a...b]$, a sub-forest is selected. Using a fixed a and increasing b allows the algorithm to have a base case (a single node) and build up to the entire sub-tree to complete the primary dynamic programming matrix.

One key aspect of this algorithm is that the cost of adding, deleting, or re-labelling nodes is determined by functions that the implementer can provide. As discussed below, this project implements a custom re-labelling cost, determined by the specific HMMs contained in each node. Once the criteria for costs are determined, the algorithm used a simple selection criteria:

$$forestdist[l[i_1]][i][l[j_1]][j] = \min \begin{cases} forestdist[l[i_1]][i-1][l[j_1]][j] + delete_cost(end_1) \\ forestdist[l[i_1]][i][l[j_1]][j-1] + add_cost(end_2) \\ forestdist[l[i_1]][l[i]-1][l[j_1]][l[j]-1] + forestdist[l[i]][i-1][l[j]][j-1] + rename_cost(i, j) \end{cases}$$

i_1 and j_1 are arbitrary ancestors of i and j used to enforce a meaningful range.

To improve the algorithm runtime, the nodes considered can be reduced. Since the final result, tree-to-tree distance, is constructed by subtrees, Zhang and Shasha introduce the notion of Keyroots. Effectively, keyroots are the nodes in the start and end tree where distinct subtrees (and sub-forests) can be selected. Finding them is simple: it is the root node, and any node that has a sibling to its left. It is possible to improve the algorithm by re-using additional work. Since sub-trees and sub-forests are calculated in increasing size, calculating forest edit distance can re-use the tree edit distance for the trees within. This gives updated selection criteria. When operating on a single node, that is, $l[i] = l[i_1]$ and $l[j] = l[j_1]$, we use the selection

$$forestdist[l[i_1]][i][l[j_1]][j] = \min \begin{cases} forestdist[l[i_1]][i-1][l[j_1]][j] + delete_cost(end_1) \\ forestdist[l[i_1]][i][l[j_1]][j-1] + add_cost(end_2) \\ forestdist[l[i_1]][l[i]-1][l[j_1]][l[j]-1] + rename_cost(i, j) \end{cases}$$

And otherwise, re-use the tree distance:

$$forestdist[l[i_1]][i][l[j_1]][j] = \min \begin{cases} forestdist[l[i_1]][i-1][l[j_1]][j] + delete_cost(end_1) \\ forestdist[l[i_1]][i][l[j_1]][j-1] + add_cost(end_2) \\ forestdist[l[i_1]][l[i]-1][l[j_1]][l[j]-1] + treedist(i, j) \end{cases}$$

Note that this algorithm runs in $O(n^4)$ time and $O(n^2)$ space. While the full details are provided in the analysis section, it is worth summarizing quickly. Building the array of all possible tree-to-tree edit distances requires n^2 values, as it is bounded by the number of nodes in the largest tree; and each of these values requires an array of forest-to-forest edit distance, again bounded by the number of nodes in the larger tree. $O(n^2)$ by $O(n^2)$ yields the $O(n^4)$ time complexity, but space remains $O(n^2)$, as only a single instance of the tree-to-tree table and forest-to-forest table are required at once.

74 B. Hidden Markov Model Distance

75 C. Hidden Markov Model Distance

76 D. Hidden Markov Model

A Hidden Markov Model (HMM) consists of a finite set of states X and an alphabet Y . Each state $x \in X$ has an emission probability function, given as a vector $p(y|x_i)$, that gives the probability of emitting a symbol $y \in Y$, and each state x has a transition probability function that specifies the probability of moving from the current state to any other state in X at the next observation period, also given as a vector $p(x_t|x_{t-1})$.

This model can emit a sequence of observations, $y_{1:T}$. The likelihood that a given model has produced a particular observation sequence can be computed efficiently using the forward algorithm.

A naive solution to find the probability of being in state x at time T and emitting symbol y_T , by summing the probability of all possible paths which could have emitted the entire sequence and ended in state x . Instead the forward algorithm uses the

Algorithm 2 subprocedure: find_treedist

Require: node i in T_1 and node j in tree T_2
Require: LRK_1 , LRK_2 , and $l[x]$ in scope

 1: $forestdist[0][0][0][0] \leftarrow 0$
 $\triangleright$ Not really a 4-D array: the first and third indices are fixed, so it is actually 2-D

 $\triangleright$ Base case: any sub-forest from T_1 can be reduced to the empty forest

 2: **for** end_1 in $range(l[i], i)$: **do** //inclusive range

 3: $forestdist[l[i]][end_1][0][0] = forestdist[l[i]][end_1 - 1][0][0] + delete_cost(i)$

 4: **end for**
 $\triangleright$ Base case: any sub-forest in T_2 can be constructed from the empty forest

 5: **for** end_2 in $range(l[j], j)$: **do** //inclusive range

 6: $forestdist[0][0][l[j]][end_2] = forestdist[0][0][l[j]][end_2 - 1] + insert_cost(j)$

 7: **end for**

 8: **for** end_1 in $range(l[i], i)$: **do**

 9: **for** end_2 in $range(l[j], j)$: **do**

 10: **if** $l[end_1] == l[i]$ and $l[end_2] == l[j]$ **then**
 $\triangleright$ forestdist completed

 11: $result = \min \begin{cases} forestdist[l[i]][end_1 - 1][l[j]][end_2] + delete_cost(end_1) \\ forestdist[l[i]][end_1][l[j]][end_2 - 1] + add_cost(end_2) \\ forestdist[l[i]][end_1 - 1][l[j]][end_2 - 1] + rename_cost(end_1, end_2) \end{cases}$

 12: $treedist[end_1][end_2] = result$

 13: **return** $result$

 14: **else**

 15: $result = \min \begin{cases} forestdist[l[i]][end_1 - 1][l[j]][end_2] + delete_cost(end_1) \\ forestdist[l[i]][end_1][l[j]][end_2 - 1] + add_cost(end_2) \\ forestdist[l[i]][l[end_1] - 1][l[j]][l[end_2] - 1] + treedist[end_1][end_2] \end{cases}$

 16: $forestdist[l[i]][end_1][l[j]][end_2] = result$

 17: **end if**

 18: **end for**

 19: **end for**

Algorithm 3 Forward Algorithm for HMM

1: Initialize

 2: $t = 0$

 3: transition probabilities: $p(x_t | x_{t-1})$

 4: emission probabilities: $p(y_t | x_t)$

 5: observations: $y_{1:T}$

 6: prior probability: $\alpha(x_0)$

 7: **For** $t = 1$ to T :

 8: $\alpha(x_t) = p(y_t, x_t) \sum_{x_{t-1}} p(x_t | x_{t-1}) \alpha(x_{t-1})$

 9: **Return:** $p(y_{1:T}) = \sum_{x_T} \alpha(x_T)$

85 markov property (the current state only depends on the previous state) to construct an optimal substructure. We can say that
 86 $(x_T | y_{1:T})$ only depends upon the solutions to $(x_{T-1} | y_{1:T-1})$, and the constant transition / emission probabilities applied to these
 87 subproblem solutions.

88 In algorithm 3, we present the forward algorithm. After initializing the model parameters and prior probabilities, we iterate
 89 through each time step t from 1 to T . We then update the probability of being in a given state by summing over all probabilities
 90 from the previous time step, weighted by their associated transition probabilities, and multiplying the sum by the emission
 91 probability of the current observation.

92 The final result of this step is the probability of being in each state at time T , given the entire sequence of observations up
 93 to that point. To calculate the total probability of the HMM emitting the sequence we sum over all final state probabilities.

E. Distance Calculation

To compare two Hidden Markov Models H_1 and H_2 , we generate several sequences of observations from each model. Let these sequences be labeled $h_{1_1}, h_{1_2}, \dots, h_{1_n}$ from H_1 and $h_{2_1}, h_{2_2}, \dots, h_{2_n}$ from H_2 . We approximate the Kullback–Leibler (KL) divergence in both directions and then average over the number of sequences. While many other divergence measures could be employed, such as the Jeffreys divergence, due to time constraints we only tested using KL divergence. The approximation is given by

$$D(H_1, H_2) \approx \frac{1}{2n} \left(\sum_{i=1}^n \log \frac{p(h_{1_i} | H_2)}{p(h_{1_i} | H_1)} + \sum_{i=1}^n \log \frac{p(h_{2_i} | H_1)}{p(h_{2_i} | H_2)} \right).$$

Here $p(h | H)$ denotes the probability of sequence h under model H , computed using the forward algorithm.

III. ANALYSIS

A. ZSS Complexity

The Zhang-Shasha algorithm runs with time complexity $O(n^4)$ and space complexity $O(n^2)$, where n is bounded by the number of nodes in the larger tree. More specifically, Zhang and Shasha state that the algorithm requires $O(|T_1| \times |T_2| \times \min(\text{depth}(T_1), \text{leaves}(T_1)) \times \min(\text{depth}(T_2), \text{leaves}(T_2)))$ in time and $O(|T_1| \times |T_2|)$ in space. (Zhang and Shasha, 1989) As described in the formalism section, this is due to the two-stage dynamic programming approach. Tree edit distances are computed for each subtree in T_1 to each subtree in T_2 (both clearly bounded by n , the number of nodes in the larger tree) which requires $O(n^2)$ time and space. However, for each of these calculations, further dynamic programming is required. Each sub-tree edit distance is computed via sub-forest edit distance, again, from any possible sub-forest from the T_1 subtree to any subforest in the T_2 subtree. Since the subtrees are bounded by the n , and the sub-forests bounded by the size of the subtrees, this inner calculation is also $O(n^2)$ for every one of the tree-to-tree cells which requires it. Combining the $O(n^2)$ outer calls with the $O(n^2)$ inner calls produces the $O(n^4)$ time complexity. Since only one table is needed for the outer calls, and only one inner call is active at once, the space complexity increases linearly, and $O(n^2) + O(n^2)$ remains $O(n^2)$.

More modern versions of this algorithm exist, with slightly better ($O(n^3)$) time complexity. However, these algorithms are much more difficult to implement, and Paaßen (2019) cites Pawlik and Augsten (2011) in explaining that "... the practical runtime complexity of the original TED algorithm is still competitive for balanced trees, such that we regard it as a good choice in many practical scenarios." Further, as the number of bifurcation nodes in a CM is relatively few, the number of keyroots found by the algorithm will also be relatively small, further reducing the work needed in our specific scenario.

B. HMM Complexity

The forward algorithm has time complexity $O(nm^2)$, where n is the length of the observation sequence and m is the number of hidden states. At each time step, the algorithm must compute the probability of transitioning from all states at time $t - 1$ to all states at time t :

$$\alpha(x_t) = p(y_t, x_t) \sum_{x_{t-1}} p(x_t | x_{t-1}) \alpha(x_{t-1}).$$

The emission probability $p(y_t | x_t)$ acts only as a multiplicative constant in this computation, so the dominant cost arises from the double summation over states, giving m^2 . Since this computation is performed for each of the n time steps, the overall time complexity is $O(nm^2)$.

The label distance algorithm requires we use the HMM to generate many random, but statistically likely, sequences, each of indeterminate length. As this is a random process, a worst-case time complexity cannot be specified without fixing the length of the generated strings. However, if the length of a string is known, the complexity of its generation is linear in that length. Thus, the cost of string generation can be folded into the overall complexity of the forward algorithm, yielding the same $O(nm^2)$ bound when combined with the dynamic programming steps.

IV. IMPLEMENTATION

The algorithm is implemented in Python. We make use of the `hmmlearn` library to store Hidden Markov Models, to emit sequences of observations, and to apply the forward algorithm for likelihood calculations. For the ZSS algorithm, we employ the `zss` package to perform the Zhang–Shasha tree edit distance, augmented with our custom distance metric to tailor the comparison to our models.

A. Node HMM Model Augmentations

In contrast to the formal description of the algorithm, the concrete implementation requires structural adjustments to the Hidden Markov Models. Specifically, each model is augmented with a single entry state and a final exit state. The reason for the initial incoming transition is that, while we represent each node as an HMM, this is a simplification. A node may have several incoming and several outgoing transitions. To account for this, these incoming and outgoing transitions are collected and normalized into one incoming and one outgoing state, with transition probabilities summed and normalized. This ensures that every sequence begins from a unique entry point and terminates in a unique exit point.

As well, in the concrete models there are hidden states that are silent. The forward algorithm cannot directly handle the potentially infinite paths that arise from such ϵ -transitions, so these states must be removed. Eliminating silent states while preserving the transition and emission probabilities of the non-silent states requires an $O(n^3)$ preprocessing step, where n is the number of states in the model. This transformation is performed once for each node.

Alternatively, we could have chosen to force every silent state to emit a self-identifying character. We believe this modification would shift the comparison between HMMs toward a more structural rather than emission-based distance measure. Further research could evaluate the effect of this approach.

B. Distance Metric Augmentations

Because the ZSS algorithm must also account for insertions and deletions, we require a consistent rule for comparing a node to its absence. We handle this by comparing the observed sequences emitted from the node, to the probability that it emits the empty string.

$$D(H, \emptyset) \approx \frac{1}{2n} \left(\sum_{i=1}^n \log \frac{p(h_i | H)}{p(\emptyset | H)} + \sum_{i=1}^n \log \frac{p(\emptyset | H)}{p(h_i | H)} \right).$$

Here $\emptyset$ denotes the empty string, and h_i are the observed strings generated from the model H .

Additionally, certain nodes in the concrete model cannot be represented internally as Hidden Markov Models. Specifically, the BIF and END nodes are treated as structural markers rather than probabilistic models. Any time these nodes are compared to like named nodes they will return a log-odds of 0.0 (perfect match) and a default high value otherwise. For this test we chose -5.0 odds.

C. Memoization

To reduce redundant computation, many values are saved during parsing and model construction. All sample strings required for label comparison are generated once and stored, together with the probability that the generating model produced it, given by the forward algorithm.

Furthermore, when comparing two CM's with the ZSS algorithm, the distance between nodes is save after the first computation. Subsequent distance measures are retrieved directly from this cache, eliminating repeated evaluations and improving overall efficiency.

V. EVALUATION

Having implemented this approach for comparing covariance models, we need a way to evaluate its performance. We explore two methods to evaluate how well our approach compares covariance models. First, is an extremely naïve statistical analysis that will use the RNA family's identified clan to compare within- and between-group similarity. Second, is a more nuanced approach that utilizes concrete sequence comparison as a base truth for comparing the models.

Due to time constraints, we only fully implemented the first comparison method. We will still describe the second method as it is useful for context and potential future work.

For this first method, we utilized the clan classification given to some groups of RNA families. Naïvely, we assume that the distance of covariance models within a clan is on average smaller than the distance of covariance models between clans. However, using clans to perform within- and between-group analysis is flawed for several reasons. Primarily, the clans are not strictly organized based on structure. As our approach implements a structural comparison between covariance models, comparing the distance of models within clans and models between clans will not necessarily be representative. Another flaw being the variability in the number of identified RNA families in a clan. As we have an uneven distribution of potential within- and between-clan comparisons, our comparison will need to be resilient to that bias in the dataset. Despite these flaws, we still attempt this comparison method.

Permutational multivariate analysis of variance (PERMANOVA) is a good statistical test for this comparison. Compared to ANOVA, which compares the means of groups, PERMANOVA uses a distance measurement between values. This matches perfectly with our need to evaluate our distance comparison and mitigates the impact of our disparate group sizes.

We arbitrarily selected 40 covariance models from 10 clans for use in our evaluation. This gives 780 combinations of families to compare. On an M4 MacBook Pro with 24GB of RAM, this took approximately 3 minutes to complete. Given the distance

from these combinations, we construct the distance matrix necessary for PERMANOVA. Running this statistical test resulted in a test statistic of approximately **15** and a p-value of approximately **0.001**. The test statistic describes the similarity of the within- and between-group comparisons. We will discuss the implications of these results in Section VI.

Our second method of evaluation is to use the comparison of RNA sequences belonging to an RNA family as a baseline truth to measure our comparison of covariance models. As the comparison of RNA sequences is a structural comparison, the flaw of using clans would not be an issue. If our method of comparing covariance models is an effective measurement, we would expect there to be a correlation for the distance measured between RNA sequences of different families and distance between covariance models. Unfortunately, due to time constraints and our inexperience in bioinformatics, we were unable to perform this analysis.

VI. DISCUSSION

From our evaluation of our comparison approach using PERMANOVA, we have found that using the Zhang-Shasha tree edit distance algorithm and our HMM distance cost function produced results that were statistically significant, but not necessarily meaningful. The test statistic of approximately **15** means that there is significant difference in the within- and between-group distances. Further, as our p-value is **0.001**, our results are likely statistically significant.

There are threats to our approach. While Zhang-Shasha is a competitive algorithm for tree edit distance of ordered and labeled trees, it is possible that our chosen ordering of nodes from the covariance model negatively impacted our results. It is also possible that the cost function we implemented to determine the cost of deleting, inserting, and re-labeling nodes was not optimal. Our HMM fragment to HMM fragment is a probabilistic process involving produced sequence likely hood and could be potentially tuned for better or worse results. Without a rigorous method of evaluation, these individual aspects of our approach are difficult to properly evaluate and potentially tune.

Another potential threat is our small dataset. We only considered 40 covariance models and 10 clans. At the time, there are 4,170 RNA families and 147 clans on Rfam, meaning that there is a much larger dataset we could reach for.

Using all 4,170 RNA families (assuming each family on Rfam has a covariance model) would mean just under 8.7 million comparisons to make. It is unlikely we could have completed our approach on the entire dataset in a reasonable time frame using the consumer hardware available to us. Despite this limitation, it is possible that the PERMANOVA test on this larger dataset could produce different results. However, due to the previously discussed issue with using clans as a category, this approach would likely still be flawed.

The second method of using correlation of RNA sequences comparison to covariance model comparison is a much more promising avenue for fairly evaluating our approach. RNA sequence comparison or alignment is a well studied problem in bioinformatics, with a number of approaches and techniques. Given enough time, we likely could have repurposed an existing tool for this. We regret that we were unable to conduct this evaluation in time to include in this paper.

One benefit of our approach is the actual run time for comparing two models. While we have no concrete data for any one particular run, being able to perform 780 comparisons in approximately 3 minutes on consumer hardware represents a reasonable run time. Due to our implementation being written in Python and utilizing no amount of parallelization or caching between comparisons, it is likely that there are significant performance gains to be made. As each comparison is independent of any other comparison, parallelization would be an easy optimization that would likely see time improvements, even on consumer hardware.

Overall, we find our approach to have significant potential despite the potential flaws in our evaluation. Further work is needed to verify our approach and confirm the promising results we have.

Originally we presented a negative result to this approach. This was due a bug in the HMM comparison.

VII. RELATED WORK

VIII. CONCLUSION

We present our approach for structural comparison of RNA family covariance models using tree edit distance and HMM to HMM comparison. The Zhang-Shasha algorithm is a competitive and robust method of computing the distance between two ordered and labeled trees in a reasonable time complexity for the task. Our HMM to HMM comparison leverages the internal structure of nodes within a covariance model to produce a cost function that can be used in the tree edit distance algorithm. Together, we find these two approaches to be an interesting, if not promising, way to compare covariance models.

Despite the flaws in our evaluation of this approach, we believe that utilizing a different method of evaluation would allow us to better scrutinize this approach and present ways to improve the details of our HMM to HMM comparison. It is unfortunate that we were unable to perform a better evaluation before the writing of this paper. This leaves significant future work open for developing and evaluating this approach.

Ultimately, we find that this approach was a worthwhile first attempt at using tree edit distance for comparison of covariance models.

IX. DISCLAIMER

239

240 This report was written with assistance from an llm working in a strictly editorial capacity. All content and ideas are the
241 original work of the authors.